

AFRILANG Stdlib

std/c/parseuri

Bibliothèque AFRILANG `std/c/parseuri` (niveau complex, catégorie complex). Elle expose 7 fonction(s) :
scheme, host, pa

```
`import "std/c/parseuri"` · `use parseuri`
```

- `scheme(uri text) ? text`
- `host(uri text) ? text`
- `path(uri text) ? text`
- `query(uri text) ? text`
- `fragment(uri text) ? text`
- `port(uri text) ? number`
- `isValid(uri text) ? bool`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/parseuri` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : `scheme`, `host`, `pa`

```
`import "std/c/parseuri"` · `use parseuri`  
- `scheme(uri text) ? text`  
- `host(uri text) ? text`  
- `path(uri text) ? text`  
- `query(uri text) ? text`  
- `fragment(uri text) ? text`  
- `port(uri text) ? number`  
- `isValid(uri text) ? bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/parseuri"  
use parseuri
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? scheme(uri text) ? text  
? host(uri text) ? text  
? path(uri text) ? text  
? query(uri text) ? text  
? fragment(uri text) ? text  
? port(uri text) ? number  
? isValid(uri text) ? bool
```

4. Exemples d'utilisation

```
import "std/c/parseuri"  
use parseuri  
  
create result = scheme(/* args */)   
say result  
  
create result = host(/* args */)   
say result  
  
create result = path(/* args */)   
say result  
  
create result = query(/* args */)   
say result  
  
create result = fragment(/* args */)   
say result  
  
create result = port(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez `import "std/c/parseuri"` en tête de fichier.
- ? Lancez `afrilang check` avant de livrer.
- ? Combinez avec les modules stdlib de la même catégorie.
- ? Voir /stdlib/ pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module parseuri
```

```
function scheme(uri text) returns text
end

function host(uri text) returns text
end

function path(uri text) returns text
end

function query(uri text) returns text
end

function fragment(uri text) returns text
end

function port(uri text) returns number
end

function isValid(uri text) returns bool
end
end
```

ENGLISH

1. Overview

AFRILANG library ``std/c/parseuri`` (tier complex, category complex). Exports 7 function(s): `scheme`, `host`, `path`, `query`, `fr`

```
`import "std/c/parseuri"` . `use parseuri`  
- `scheme(uri text) ? text`  
- `host(uri text) ? text`  
- `path(uri text) ? text`  
- `query(uri text) ? text`  
- `fragment(uri text) ? text`  
- `port(uri text) ? number`  
- `isValid(uri text) ? bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/parseuri"  
use parseuri
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? scheme(uri text) ? text  
? host(uri text) ? text  
? path(uri text) ? text  
? query(uri text) ? text  
? fragment(uri text) ? text  
? port(uri text) ? number  
? isValid(uri text) ? bool
```

4. Usage examples

```
import "std/c/parseuri"  
use parseuri  
  
create result = scheme(/* args */)   
say result  
  
create result = host(/* args */)   
say result  
  
create result = path(/* args */)   
say result  
  
create result = query(/* args */)   
say result  
  
create result = fragment(/* args */)   
say result  
  
create result = port(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/parseuri"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module parseuri
```

```
function scheme(uri text) returns text
end

function host(uri text) returns text
end

function path(uri text) returns text
end

function query(uri text) returns text
end

function fragment(uri text) returns text
end

function port(uri text) returns number
end

function isValid(uri text) returns bool
end
end
```